
Marvin User Manual

Complete guide to the local chat and coding workstation

Application version	1.6.0
User manual	2026

Contents

1. What Marvin Is	3
2. Requirements and Installation	3
3. Desktop Interface Tour	5
4. Models, KV Cache, Context, and Thinking	6
5. Sending Prompts, Steering, Stop, and Progress	7
6. Work Modes	8
7. Projects and Workspaces	9
8. Chat Management	10
9. Context, Compression, and Pinned Files	11
10. Three-Layer Memory	12
11. Optional Skills	12
12. Internet and Research	13
13. Writing and Document Export	14
14. Development Workflow	14
15. Computer Control	16
16. Autonomy and Confirmations	17
17. Terminal UI and CLI	17
18. Files, Storage, Backup, and Logs	18
19. Troubleshooting	19
20. User-Facing Tool Reference	20
21. Practical Request Examples	23
22. Operational Notes and Limitations	23
23. Extended Workspace Operations	24

1. What Marvin Is

Marvin is a private Windows desktop application for running local language models as a general chat assistant, research assistant, writing partner, coding agent, and computer operator. Model inference runs on your workstation through `llama.cpp`; conversations and project files remain on local storage.

The application is designed for one user and one active model. It does not use parallel model agents. The active model can, however, run independent read-only tools in parallel and can leave long commands running in the background.

NOTE: Model inference is local. When the model uses `web_search` or `web_fetch`, the application makes ordinary internet requests to search engines and public web pages. Disable web access in `config.yaml` or avoid Research/Web requests when you require a fully offline session.

Main capabilities

- General conversation without coding behavior.
- Multi-source web and document research with a final synthesis.
- Writing and revision of scripts, articles, reports, treatments, and other documents.
- Local project development with file, Git, shell, test, image, and rollback tools.
- Screenshot-based control of Windows applications.
- Persistent projects, multiple chats per project, three-layer memory, pinned files, and optional skills.
- English and Czech interface, switchable without losing the active chat.

What the model can see

Selecting a project gives the model access to that directory through tools. It does not place every file body in the context window. The model receives a compact project map and reads individual files when useful. Pinned files and the three memory layers are exceptions: they are deliberately included as persistent context.

2. Requirements and Installation

Recommended workstation

Component	Recommended configuration
Operating system	Windows 11, 64-bit
GPU	NVIDIA RTX 5090 with 32 GB VRAM
Driver	Current NVIDIA driver compatible with the bundled CUDA build
System RAM	Enough for Windows, model mapping, projects, and tools; 64 GB or more is comfortable
Free disk space	At least 65 GiB for all models, runtime, and working data
Python	64-bit Python 3.12, installed separately; enable Add Python to PATH
WebView / browser	Microsoft Edge WebView2 and Microsoft Edge, normally present on Windows 11

Other NVIDIA GPUs may work, but the supplied contexts and quantizations were tuned and tested for a 32 GB RTX 5090. Lower-VRAM cards require smaller contexts, lower quantization, fewer GPU layers, or CPU offload.

NOTE: Python 3.12 is a required external prerequisite for every installation. Setup.exe creates an isolated virtual environment, but it does not contain or install Python itself. Install the 64-bit Python 3.12 release from python.org and select **Add Python to PATH** before installing Marvin.

Installing with Setup.exe

1. Install 64-bit Python 3.12 and enable **Add Python to PATH** in the Python installer.

2. Run `Marvin-Setup-<version>.exe`.
3. Choose English or Czech in the installer. English is the default.
4. Choose whether to create a desktop shortcut.
5. Keep the post-install environment/model setup selected on the first installation.
6. Allow the console setup to complete. It creates the Python virtual environment, installs dependencies, downloads `llama.cpp`, and downloads the configured models.
7. Start **Marvin** from the Start Menu or desktop.

The default installation directory is:

```
%LOCALAPPDATA%\QwenHarness
```

The complete download is approximately 59 GiB and includes Qwen Q4, Qwen Q5, Ormith Q5, and the vision projectors. Interrupted Hugging Face downloads can normally be resumed by running **Set up environment and models** again from the Start Menu.

First launch

The desktop launcher checks the Python environment, dependencies, `llama.cpp`, and the required model files. Missing components trigger the setup workflow. Once installed, ordinary launches do not download the models again.

Starting the desktop application opens the native WebView window, starts the Web UI, and starts the selected `llama-server` model when required. Closing the desktop application stops its Web UI and model server and releases GPU memory.

Updating

Install the new `Setup.exe` over the existing installation. User-created data, models, sessions, projects, user skills, and saved UI state are preserved. Changed application files and bundled system skills are updated. Models are downloaded again only when a configured model file is missing.

Uninstalling and removing all data

Windows uninstall removes installed application files. Large runtime data and user-generated data may remain so that an update or reinstall does not require downloading everything again.

To remove everything after uninstalling, inspect and delete the remaining installation directory only when you no longer need:

- `runtime\models` - GGUF model files.
- `sessions` - chat history, attachments, research ledgers, and exports without a project.
- `projects` - projects created inside the application.
- `memory` - global and work-mode memory.
- `user-skills` - personal skills.
- `projects.json` - project registry.

WARNING: Deleting these directories is permanent. Back up projects, sessions, memory, and user skills first.

Running from the source checkout

```
py -3.12 -m venv .venv
.venv\Scripts\python scripts\setup_env.py --model all
npm --prefix frontend ci
npm --prefix frontend run build
.venv\Scripts\python qwen_app.py
```

For terminal use, run `run_cli.bat` or `.venv\Scripts\python tui.py`.

Source development requires Node.js to build the frontend. A normal Setup.exe installation already contains the built frontend and does not require Node.js.

3. Desktop Interface Tour

The workspace has project/chat navigation on the left, the conversation in the center, and a closable detail panel on the right. The prompt stays at the bottom of the conversation. At smaller widths, the detail panel becomes a drawer.

Navigation and work modes

The project selector opens the most recent chat of that project. **No project** opens an independent chat. Selecting another chat never rebinds the task that is already running elsewhere.

Work modes always appear in this order: **Discussion, Research, Writing, Development, Computer**. The mode belongs to the chat. A running request retains the configuration with which it started.

Use **New chat** to create a conversation. Type into the title above the chat and press Enter to rename it. The chat's three-dot menu contains Move, Undo last turn, Branch, Export, and Delete. Selecting a destination moves the chat and updates its project context. Stop the chat's task before moving or deleting it.

The three-dot menu beside the project selector creates a new project, attaches an existing folder, or deletes the selected project and its folder. Search in the left sidebar searches saved conversations.

Prompt, Attach, and image previews

The visible **Attach** button is always below the prompt. It opens the disk file picker and accepts multiple files. Images can also be dragged into the workspace or pasted from the clipboard with Ctrl+V. Ordinary text pasting still works.

Every pending image has a real thumbnail and an individual remove button. Removing an image leaves the prompt and other attachments intact. Clicking a thumbnail enlarges the original. After sending, thumbnails belong to the user message and remain available after reopening the chat.

PDF, DOCX, XLSX, CSV, Markdown, and text files can also be attached. The model receives their file paths and uses the document tools to read them. A text-only model cannot inspect image content; choose a vision-capable model when required. Draft text and uploaded attachments are retained when a request is rejected.

Enter and Ctrl+Enter send; Shift+Enter inserts a new line. The composer clears after the service accepts the identified message. Draft text is stored per conversation.

Running tasks and the message queue

While a task is active, choose **Clarify now** to steer it or **After completion** to queue a separate request. A task in a different chat keeps running; a new request waits for the single model worker. Queued text can be edited, and queued messages can be cancelled.

Stop task works during prompt processing, reasoning, visible text, tool preparation, and synchronous actions. It may finish the current visible sentence briefly; it does not wait for a whole reasoning block. Already completed work and visible partial text are retained. Stop also pauses the pending queue; **Resume queued messages** starts it again. **Continue** resumes the interrupted task.

Refreshing or reconnecting the browser does not own or cancel the model run. After a full application restart, interrupted work is available to continue. If a tool result was not saved before a crash, the model is told to inspect the actual outcome instead of blindly replaying the action.

Results, Progress, and Context

Results lists created documents and changed files, actual recorded checks, research exports, and restore points. Open a file to preview it, open its folder, or download it. HTML results can be previewed as a local application; supported programs also offer **Run**.

Progress shows the task plan, durable operational notices, changed files, background processes and their output, and the isolated browser state. Background processes are stopped separately when appropriate.

Context shows the estimated current input size and physical context limit, measured usage from the last completed model request when available, three active memory layers, pinned files, and loaded skills. Pin or unpin individual files, clear pins, compress earlier context, or hand off to a new conversation.

During generation, the activity line distinguishes preparation, thinking, writing, tool-call preparation, and execution. It includes elapsed time and an explicitly approximate live token count.

Settings

Click the settings icon or the model status in the top bar. Settings are grouped into:

- **Model and device:** model, KV profile, vision capability, GPU memory choice, runtime diagnostics, Start / Stop / Restart.
- **Behavior:** autonomy and default handling of messages sent during work.
- **Memory and skills:** edit global, mode, and project memory; read or use a skill; design a skill; open user and project skill folders.
- **Data and backups:** export/import a complete project, import chat JSONL, create/select/verify the local runtime backup, and monitor maintenance operations.
- **Appearance and language:** dark, light, or system appearance; spacing; English or Czech.
- **Help and manuals:** both PDF manuals and the slash command reference.

Thinking depth also stays beside the prompt because it can change between questions. Model settings selected while a request runs apply to subsequent requests. Language and theme changes update the interface without discarding the conversation.

Project decisions

Project decisions holds proposed, accepted, and retired decisions. Add or edit a decision, choose its status, and open its originating conversation. Only accepted decisions are included automatically in project context. They supplement the three memory layers; a proposal is not silently treated as an accepted fact.

4. Models, KV Cache, Context, and Thinking

Installed model profiles

Model	Typical role	Weight quantization	KV choices	Tested context
Qwen 3.8 27B Q5	Main high-quality model	Q5	F16 or Q8	F16 96k; Q8 192k
Qwen 3.8 27B Q4	Faster and largest-context option	Q4	F16 or Q8	F16 128k; Q8 256k
Ornith 1.5 35B-A3B Abliterated Q5	Very fast optional reasoning MoE	Q5	Q8 fixed	Q8 128k

The default new-installation profile is Qwen Q5 with Q8 KV and a 192k context. The application remembers the last selected model and each model's KV choice.

Additional configured model profiles include Qwen IQ3 for smaller GPUs and Nemotron Q4/Q5. Nemotron is text-only. The model list shows whether each file is installed, and the KV dropdown shows the profiles configured for that model. These are configuration choices, not a claim that every profile has been benchmarked on every PC.

Choosing Qwen Q5 or Q4

Use Qwen Q5 for serious development, architecture, difficult reasoning, and final-quality writing. Use Qwen Q4 when speed or the full 256k context matters more than the small quality advantage of Q5.

Q5 with Q8 is a strong general default. Change to F16 when maximum KV precision matters and 96k tokens are enough.

Ornith

Ornith is a Mixture-of-Experts model and generates very quickly because only part of its weights is active per token. All model weights must still fit in VRAM. In practical application development it may produce weaker results than dense Qwen despite its speed.

Ornith supports thinking on/off natively. The intermediate depths are prompt-guided rather than native `reasoning_effort` values:

- `xhigh` adds deliberate planning, architecture, edge-case, and verification guidance.
- `medium` adds moderate reasoning guidance.
- `low` uses Ornith's ordinary thinking behavior.
- `off` disables thinking.

KV cache precision

KV cache stores the model's attention history. It is separate from the quantization of the model weights.

Setting	Benefit	Tradeoff
F16	Highest KV precision	Approximately half the context of Q8 on the same GPU
Q8	Much larger context with lower VRAM use	Small precision tradeoff in the attention cache

Changing KV precision restarts the server. It does not delete or reset the chat.

Thinking levels

For Qwen, `xhigh`, `medium`, and `low` are sent through the model's native reasoning-effort control. `off` disables the thinking block. The setting can be changed between prompts in the same chat.

Reasoning tokens consume generation time and temporary context while the response is being produced. The UI shows a live estimate. Reasoning is saved with the answer and can be expanded in history. Persistent context estimates include reasoning sent back to the model; measured usage refers to the last completed request.

No artificial task or output limit

The production agent has no step limit and no application-level output-token cap. It continues until the task is complete, the model reaches its physical context boundary, a tool/server fails, or the user presses Stop.

5. Sending Prompts, Steering, Stop, and Progress

Ordinary prompt

Write the request in natural language. The model automatically receives the active work mode, project, memory layers, current project snapshot, pinned files, and available skill metadata.

For best results, state the result you want, important constraints, and how completion should be verified. The model is allowed to choose its process unless your prompt specifies it.

Image attachment

Use **Attach**, drag and drop, or Ctrl+V to add one or more images. Supported formats are BMP, GIF, JPEG, PNG, and WebP. The most recent images remain in active context; older image references stay in history but may be omitted from later API requests.

Images can be photographs, diagrams, screenshots, UI references, error messages, or visual source material. Qwen and Ornith use their matching vision projector automatically.

Steering a running task

Sending another message while the model is working does not merely queue a second independent task. It steers the active one:

1. The clarification is accepted immediately and the composer clears.
2. The current generation stops after the nearest completed sentence or safe chunk.
3. The finished partial response remains visible and saved.
4. The clarification is inserted into the same task.
5. The model resumes with the updated instruction.

Use steering for corrections such as "keep the existing camera behavior", "do not change the API", or "also export this as PDF".

Stop

Stop directly signals the independent run controller, including while no stream bytes arrive. It requests a graceful generation stop and normally allows the nearest sentence to finish. It also cancels the currently awaited browser operation or synchronous `run_command`, terminating that command's process tree. Finished partial text and captured command output are retained. A new prompt starts cleanly afterward.

Long-running operating-system processes have their own termination control in **Long-running operations**. Stopping generation does not necessarily stop a background process that was already launched; use the process panel when required.

Live progress

The live chat distinguishes:

- Thinking/reasoning.
- Visible response text.
- Preparation of large tool arguments, such as code for `write_file`.
- Execution of tools, commands, tests, searches, and file operations.
- Seconds without new tokens.

For example, while a large program is being generated the UI can show that content for a named file is growing instead of appearing frozen.

6. Work Modes

Each chat stores its work mode. Changing to another chat restores that chat's mode and project context.

Discussion

Use for ordinary conversation, analysis, learning, brainstorming, planning, and non-programming questions. Discussion does not inject coding procedures. It can use memory, internet, project documents, skills, pinned files, and document export.

Research

Use for web research, literature or market investigation, comparison, and evidence synthesis. Research creates a persistent plan and ledger, keeps all sources regardless of perceived trustworthiness, records contradictions, and produces a final synthesis.

Writing

Use for scripts, articles, reports, treatments, notes, and document revision. Writing can read/write project files, use web and documents, view images, create checkpoints, and export Markdown/DOCX/PDF. It does not expose Git and shell tools unless you switch to Development.

Development

Use for programming and repository work. The model receives an automatic project map and can read/search/write files, apply atomic patches, use Git, run shell commands, start tests, inspect images, maintain a task checkpoint, and roll changes back.

Computer

Includes Development tools plus screenshot, mouse, keyboard, and scrolling controls. Use it when the model must interact with a GUI application rather than only project files.

NOTE: Work modes are capability profiles, not separate model instances. Switching mode does not load another model.

7. Projects and Workspaces

No project

Choose **No project** for conversations that should not access a project folder. Chats without a project appear in their own list. Exports are stored in the chat session directory.

Creating a managed project

1. Open the three-dot project menu.
2. Choose **New**.
3. Enter a project name and confirm.

The application creates a new folder under its projects directory, registers it, creates project memory when needed, and uses it as the workspace.

Attaching an existing folder

Choose **Attach**, select an existing directory, and the application registers it as a project without copying its files. The folder itself remains in its original location.

Selecting a project

Select the project in the project dropdown. Relative file paths used by tools resolve inside this workspace. The project chat list updates to show chats whose metadata points to that project.

Multiple chats per project

A project can contain any number of chats. Each chat has independent history, work mode, context compression, pinned files, research ledger, and unfinished-task state. They share the physical project directory and project memory.

Moving a chat

In the chat's three-dot menu, open **Move to project** and select a target project or **No project**. The move happens immediately. The chat's workspace, project memory, document library, and model context are reconfigured to the target.

Deleting a project

1. Select the project.
2. Open the three-dot project menu.
3. Click **Delete project + folder**.
4. Read the exact path shown.
5. Click again within eight seconds to confirm.

Deletion removes the project registry entry, every chat belonging to the project, the project folder, and all files inside it.

WARNING: Attached external folders are also deleted from disk when you confirm project deletion. This is not a detach operation. Critical paths such as the application root, projects root, filesystem root, and home directory are blocked, but you must still verify the displayed path.

If a folder was removed outside the application, the project is marked missing. Reattach or remove the stale project entry as appropriate.

8. Chat Management

New chat

New chat creates a transient empty chat in the current project/mode. It is written to disk only after the first user message.

Delete chat

Click **Delete**, then click again within six seconds. The session folder, messages, attachments, research ledger, task state, and session-local exports are removed. Project files are not affected.

Rename

Type a title in the rename field and press Enter. The chat lists and search index update.

Retry

Retry keeps the last user prompt, removes its answer and any compression state created after it, and asks the model to answer again.

Undo

Undo removes the last complete user/assistant round from the chat. It does not roll back project files. Use **Revert task changes** for filesystem rollback.

Fork

Fork creates a new chat branch ending at the last user prompt. It copies relevant image attachments, project assignment, work mode, and pinned-file list. The original remains unchanged.

Search all chats

Open **Search all chats**, enter a word or phrase, and select a result. Search uses a persistent SQLite full-text index and covers saved chats across projects.

Export and import chat

The active-chat **Export / import** group supports:

- Markdown export for human reading.

- JSONL export preserving the message structure.
- JSONL import as a new chat.

Import never overwrites the original session. Its system prompt is refreshed to the current application version.

Handoff to a new chat

Use **Context > Hand off** for a long conversation that should continue in a fresh chat. The model summarizes essential goals, decisions, state, findings, and next steps; a new chat is created with that summary while the old history remains intact.

9. Context, Compression, and Pinned Files

Context indicator

The status and **Context** panel show estimated tokens, visible versus total messages, images, compression state, and pinned files. The detailed breakdown separates conversation/attachments, current project context, and tool definitions, so the displayed total includes tool-schema overhead.

Automatic project snapshot

In Development, the model receives a compact map of the current workspace, file types, key entry points, directories, and top-level Python symbols. It is assembled as a temporary tail for the current request and is not stored in chat history, so stale snapshots do not accumulate and the earlier reusable prefix remains unchanged.

Discussion, Research, and Writing use a project document catalog instead of a coding repository map.

The harness also discovers `AGENTS.md`, `QWEN.md`, and `CLAUDE.md` from the project root down to directories containing files currently being inspected or changed. Root guidance applies broadly; deeper files are more specific. These documents guide the model but do not override the current user request.

Pinning a file

Open **Context** and choose **Pin file**, or tell the model to pin a path. The complete text is refreshed into the context at the start of subsequent tasks in that chat.

Good pinned files include architecture decisions, an active specification, a handoff, or project rules that matter throughout the conversation.

Limits:

- Maximum 10 pinned files.
- Approximately 40,000 characters in total.
- Pinning belongs to one chat; a fork copies the pin list, a separate new chat does not.

Do not pin ordinary source files that the model can read on demand, large logs, or memory documents already injected automatically.

Manual and automatic compression

At approximately 85% of the selected model context, the harness summarizes older messages. The visible chat history is never deleted; only the model's request view changes to system prompt + summary + recent messages.

Use **Context > Compress** to compress earlier. If the model reports an overflow, the agent performs one automatic compression-and-retry cycle. A second overflow is reported instead of looping forever.

10. Three-Layer Memory

Every task receives up to three complete memory documents:

Layer	Scope	Typical content
Global	Every mode and project	User preferences, universal facts, stable conventions
Work mode	All chats using one mode	Research style, writing preferences, development conventions
Project	One workspace	Decisions, terminology, paths, project-specific facts

Locations:

```
memory\GLOBAL.md
memory\MEMORY.md           (Development mode legacy/default)
memory\modes\discussion.md
memory\modes\research.md
memory\modes\writing.md
memory\modes\computer.md
<project>\QWEN_MEMORY.md
```

Open the active memory files in **Settings > Memory and skills**. You can edit them directly or say, for example:

```
Remember globally that I prefer concise Czech summaries.
Remember for Research that every final answer must preserve contradictory findings.
Remember for this project that the delivery folder is D:\Exports.
```

The model chooses global, mode, or project storage through `save_memory`. Moving a chat to another project immediately changes its project-memory layer.

11. Optional Skills

Skills are reusable `SKILL.md` guidance packages. They do not run a second agent and do not override the user. Only each skill's name and trigger description are included in ordinary context; the body is loaded through `read_skill` when it clearly helps.

Skill precedence

1. Project skill: `<project>\.qwen-skills\<name>\SKILL.md`.
2. User skill: `user-skills\<name>\SKILL.md`.
3. Bundled system skill: `skills\<name>\SKILL.md`.

A higher layer with the same name replaces the lower one.

Bundled skills

Skill	Purpose
systematic-debugging	Reproduce, trace, test hypotheses, and fix the real cause.
architecture-options	Compare meaningful architecture options without overriding requested constraints.
implementation-verification	Proportionate requirement, artifact, test, and release verification.
performance-investigation	Measurement-first latency, throughput, memory, and bottleneck analysis.
research-synthesis	Complete synthesis that retains contradictions and all relevant sources.

Adding a user skill

Open **Settings > Memory and skills > User skills** and create:

```
user-skills\my-skill\SKILL.md
```

Minimum format:

```
---
name: my-skill
description: What this helps with and when the model should load it.
---

# My Skill

Flexible guidance, references, examples, or workflow suggestions.
```

The catalog refreshes automatically. Keep the description specific because it is the model's trigger. Put project-only skills in `.qwen-skills` inside that project.

12. Internet and Research

Ordinary web use

All work modes can use `web_search` and `web_fetch` for current information, public documentation, error messages, and web pages. `web_fetch` handles HTML and can extract text from supported PDF and DOCX downloads.

The default search backend is Google; `config.yaml` can select Google, Bing, or automatic fallback. Fetching is read-only HTTP/HTTPS.

Research workflow

Research mode automatically:

1. Records the question.
2. Creates a persistent research plan before searching.
3. Stores every search query and candidate link.
4. Stores the full readable content of fetched web/local sources.
5. Tracks coverage and source IDs.
6. Keeps contradictory and minority information.
7. Produces a final human-readable synthesis.

Sources are not hidden, discarded, or ranked because the model considers them untrustworthy. The user decides credibility and relevance. The synthesis distinguishes source claims from inference and states uncertainty and missing evidence.

Research panel and exports

The **Research progress** panel shows queries, links, sources read, and status. It can export:

- Complete research ledger containing plans and source material.
- Completed synthesis as DOCX.
- Completed synthesis as PDF.

Asking to export an already completed answer does not start a second research run.

Project documents

Readable project documents include Markdown, text, RST, CSV, JSON, YAML, PDF, and DOCX. Research mode records a local document as a ledger source when it is read.

13. Writing and Document Export

Writing mode keeps the user's intent, voice, structure, and constraints. It can read and revise project files without introducing development terminology unless the user asks for it.

The model can export final text in every work mode using natural requests such as:

```
Save the final answer as PDF.  
Export this as a structured DOCX called production-report.  
Create a Markdown file with this synthesis.
```

Supported formats:

- Markdown (.md).
- Structured Word document (.docx).
- PDF with Unicode/Czech text and basic tables/inline formatting.

Output location:

- With a project: <project>\exports.
- Without a project: sessions\<chat-id>\exports.

Exports produced through project file tools participate in the current task checkpoint and can be reverted when appropriate.

14. Development Workflow

Development mode is intended for complete repository work while keeping the main UI conversational.

Project discovery

At each request, the model receives a current repository snapshot plus applicable hierarchical project instructions. It can call `repo_overview`, `project_instructions`, list or glob files, search literal text or regex, navigate declarations with `find_symbol/document_symbols`, find whole-word uses with `find_references`, and read relevant ranges before editing.

File operations

- `read_file` reads text with line numbers.
- `search_files` uses ripgrep when available and supports literal/regex, case sensitivity, and filename glob.
- `find_files` lists matching project files through a glob.
- `write_file` creates or fully rewrites a file.
- `apply_patch` performs exact atomic text replacements.
- `make_directory`, `move_file`, and `delete_file` provide structured filesystem operations. Their changes participate in task rollback.
- `view_image` brings an image file into vision context.

Isolated browser workflow

Development and Computer modes include a persistent headless Microsoft Edge session. This is separate from the user's normal browser window and is intended for testing web applications:

1. `browser_open` loads a local or public URL.
2. `browser_snapshot` returns visible text and interactive elements with refs such as `e1`.
3. `browser_fill`, `browser_click`, and `browser_press` interact through those refs.
4. `browser_select`, `browser_hover`, and `browser_scroll` cover dropdowns, hover states, and long pages.
5. `browser_upload` sets a local file input; `browser_download` stores a downloaded file under the active chat.
6. `browser_viewport` switches between desktop, tablet, and mobile layouts.
7. A new snapshot verifies DOM-visible results; stale refs are rejected.
8. `browser_console` and `browser_network` expose console messages, HTTP responses, and failed requests.
9. `browser_screenshot` attaches the rendered page to the next Qwen request for native vision inspection.
10. `browser_close` or **Browser session > Close browser** releases the Edge process.

The browser remains available across individual agent steps. Use Computer mode instead when the task concerns a native desktop application or the entire Windows screen rather than a web page.

Task plan

For substantial work the model creates an operational plan with `set_task_plan` and updates steps with `update_task_step`. **Task progress** shows the goal, pending/in-progress/completed steps, validation result, and whether the final diff was reviewed. This task ledger is stored in `task-plan.json`, survives restart and compression, and contains operational state rather than private chain-of-thought.

Task checkpoint and rollback

The first write in a task starts a change journal. **Changes in this task** lists created/modified files. **Revert task changes** restores all journaled files to their pre-task state, including removing files created by the task.

This rollback is independent of chat Undo. Chat Undo changes conversation history; task rollback changes the filesystem.

Git

The model can inspect status and diff and create a local Git commit. `git_commit` stages the paths explicitly supplied by the model or, when omitted, only files recorded by the current task journal. It never pushes automatically. Ask explicitly when you want a commit or push.

Commands and tests

Short commands run synchronously with a timeout. Their complete stdout/stderr is stored in `sessions\<chat-id>\command-logs`; the model receives a head+tail preview so errors at the end are preserved. `Stop` cancels the active synchronous command and terminates its process tree. Long commands run in the background and return a process ID. The model can poll output, send stdin, or terminate the full process tree.

`project_validation_profile` lists detected test, lint, typecheck, and build commands. `start_project_check` starts the primary command or a named check:

- This harness: `tests/test_core.py`.
- Python: `pytest/pyproject`.
- Node: `npm test` or `npm run check`.
- Rust: `cargo test`.
- Go: `go test ./...`
- .NET: `dotnet test`.

Projects can replace auto-detection with `.qwen/project.yaml`:

```
checks:
  - id: tests
    label: Full test suite
    command: npm test
    shell: powershell
    kind: test
    timeout: 900
    primary: true
```

Completed checks are recorded automatically in **Task progress**. Before a changed task finishes, the harness gives the model one non-blocking reminder when validation, plan steps, or final diff review are still missing. The model can perform the useful checks or explain why one is not relevant.

The final verification phase is guidance, not a restriction. Explicit user requirements about architecture, format, scope, or a single-file result take priority.

15. Computer Control

Computer mode adds GUI tools to the complete Development tool set.

Operation cycle

1. The model calls screenshot.
2. The screenshot is downscaled if necessary and attached to the conversation.
3. The model identifies controls in image-pixel coordinates.
4. It clicks, moves, scrolls, types, or presses keys.
5. It captures another screenshot to verify the result.

The harness maps screenshot coordinates back to the real display resolution.

Available GUI actions

- Capture primary-monitor screenshot.
- Get screen size and coordinate mapping.
- Move mouse for hover states.
- Left/right/middle click and double-click.
- Scroll up/down at an optional location.
- Type ASCII directly or paste Unicode/long text through the clipboard.
- Press keys and combinations such as Enter, Escape, Ctrl+S, Alt+F4, or Win+D.

Failsafe

Move the physical mouse pointer rapidly to the top-left corner of the primary display to trigger PyAutoGUI's failsafe and interrupt GUI actions.

WARNING: Computer mode can see the primary screen and can perform destructive actions. On-screen text can contain misleading instructions. Use Supervised autonomy for email, payments, account changes, deletion, or any sensitive workflow.

16. Autonomy and Confirmations

Autonomy controls confirmation of WRITE-risk tools. It does not change model intelligence or available context.

Level	Behavior
Supervised	Every write, shell action, and GUI action asks for confirmation. Read-only operations proceed.
Semi	The first WRITE action in a task asks; after approval the rest of that task proceeds.
Auto	WRITE actions proceed without confirmation.

There is no task-step limit in any production autonomy mode.

When confirmation is required, the chat lists the pending actions and shows **Allow** and **Deny**. Denying returns the decision to the model so it can adapt. Repeated clicks are ignored once the pending action is gone.

Read-only commands are classified conservatively. Redirection, package installation, arbitrary Python, copying, deletion, network writes, Git commit/push, and mixed command chains are treated as writes.

17. Terminal UI and CLI

The installer creates **Marvin (CLI)** in the Start Menu. It opens `run_cli.bat`, which launches the interactive terminal UI using the installed Python environment.

Commands

Command	Purpose
<code>/memory</code>	Show global, active-mode, and project memory.
<code>/model q4 q5 ornith_q5</code>	Switch model; restarts the server.
<code>/work discussion research writing development computer</code>	Select work mode.
<code>/mode chat agent computer</code>	Compatibility shortcut for legacy agent mode.
<code>/autonomy supervised semi auto</code>	Set confirmation behavior.
<code>/thinking xhigh medium low off</code>	Set reasoning depth.
<code>/ws [path]</code>	Show or set workspace.
<code>/img <path></code>	Attach an image to the next prompt.
<code>/screenshot</code>	Capture and attach the screen.
<code>/new</code>	Create a new session.
<code>/sessions</code>	List sessions.
<code>/load <id></code>	Load a session.
<code>/server status start stop</code>	Control the inference server.
<code>/help</code>	Show command help.
<code>/exit</code>	Exit the CLI.

During confirmation, `y` allows, `n` denies, and `a` allows all remaining writes in the current task. `Ctrl+C` interrupts generation.

18. Files, Storage, Backup, and Logs

Important locations

Path	Contents
runtime\models	GGUF models and vision projectors
runtime\llama	llama.cpp CUDA binaries
runtime\webui-state.json	Last model, KV choice, language, mode, workspace, and active session
sessions\<id>	Messages, metadata, attachments, task state, research, compression, exports
projects	Managed project folders created by the app
projects.json	Registered project list
memory	Global and work-mode memory
<project>\QWEN_MEMORY.md	Project memory
skills	Bundled system skills
user-skills	Personal skills preserved by the installer
<project>\.qwen-skills	Project skills

Backup recommendations

Back up at least:

- Project directories.
- sessions when chat/research history matters.
- memory.
- user-skills.
- projects.json.

The application can also create a complete reusable installation backup from files that are already present on this computer. Open **Settings > Data and backups** and select **Create backup**. Choose a parent directory on another disk when possible. The application creates a timestamped QwenHarness-Offline-Backup-* folder containing:

- Every complete model and vision-projector file currently in runtime\models.
- The installed llama.cpp and CUDA runtime from runtime\llama.
- The current model selection and requirements.txt.
- A snapshot of the Python packages already installed in .venv.
- The matching Setup.exe when it is available in the current build directory.
- manifest.json with the size and SHA-256 hash of every backed-up file.

Backup creation copies model, runtime, and Python dependency files directly from the current Marvin directory. It does not download any of them again. It needs free space approximately equal to the installed runtime and may take time because every copied file is hashed.

Installing from the offline backup

1. Install 64-bit Python 3.12 on the destination computer and enable **Add Python to PATH**. Python itself is not stored in the backup.
2. Run the Marvin Setup.exe included in the backup. If an installer was not available when the backup was created, use a matching or newer compatible Setup.exe.
3. Use either of these methods:

- When Setup.exe is inside the backup folder beside manifest.json, run it there; it detects the backup automatically.
 - Put the backup beside Setup.exe and rename it exactly to QwenHarness-Offline-Backup; the installer detects it automatically.
 - Install Marvin, open the Start Menu, run **Set up from offline backup**, and select the backup folder.
4. Normal setup first uses the standard internet sources. If a selected model, llama.cpp, or the Python dependencies cannot be obtained online, setup restores only that failed component from the selected local backup.
 5. The explicit Start Menu command **Set up from offline backup** reverses the order: it restores the backup immediately and then only obtains anything that is still missing.

Use **Use as fallback** to register a backup for future download failures, **Verify SHA-256** to check every file against its manifest, and **Clear selection** to stop using that fallback. Selecting a backup does not disable internet access. Keep Setup.exe, manifest.json, README-OFFLINE.txt, requirements.txt, python-dependencies, and payload together in the same backup folder.

The offline installation backup does not include chats, projects, memory, or personal skills. Back up those irreplaceable user files separately as listed above.

Logs

Log	Use
runtime\launcher.log	Desktop launcher and startup problems
runtime\app.log	Native app lifecycle/crash details
runtime\webapp.log	Web UI startup and Python errors
runtime\llama-server.log	Model loading, context, CUDA, inference, and timing

19. Troubleshooting

Server does not start

- Open the Server panel and try Restart.
- Confirm the selected model exists in runtime\models.
- Read runtime\llama-server.log.
- Check NVIDIA driver/GPU availability.
- Stop another process using port 8080.
- If VRAM allocation fails, close other GPU applications, select a smaller model/context, or use the tested profile.

Web UI port is occupied

The launcher searches for an available nearby Web UI port. If another Marvin Web UI already owns the configured port, the launcher reuses it. See runtime\launcher.log for the selected URL.

Model or projector is missing

Run **Set up environment and models** from the Start Menu. Existing complete files are skipped. The downloader resumes supported incomplete downloads.

Model is slow after a long conversation

- Check context usage.
- Keep Qwen Q5/Q8 for 192k or switch Q4/Q8 for 256k.
- Use manual compression or Hand off.

- Avoid repeatedly changing early persistent memory during one long task because a changed system prefix must be evaluated again.

Context overflow

The agent automatically compresses and retries once. If it overflows again, manually compress, hand off to a fresh chat, choose a larger context profile, or remove unnecessary pinned files/images.

Interface appears frozen

Read the live activity line. The model may be thinking, generating a large tool call, writing a file, running a test, or waiting for a long process. If there is no server activity for an extended period, the harness detects a stale stream and reports it. Use Stop when appropriate.

Isolated browser does not start

- Microsoft Edge must be installed in its standard Windows location.
- Run the environment repair/update step after upgrading so Python Playwright is installed.
- The isolated session is headless; it does not open or reuse the user's normal Edge/Chrome window.

Stop did not end an operating-system process

Stop ends model generation. Open **Long-running operations** and terminate the process separately.

A task was interrupted by restart

Open **Unfinished task** and choose **Continue task**. Pending confirmations, partial visible text, journal state, and persistent process metadata are restored when possible.

Project folder is missing

The project list marks missing directories. Reattach the correct folder, move affected chats, or remove the stale project. Be careful: confirmed project deletion removes the folder and chats rather than merely unregistering it.

Where is an exported PDF or DOCX?

- Project active: <project>\exports.
- No project: sessions\<chat-id>\exports.
- Research panel synthesis export: the selected file is also presented by the UI.

Language did not fully change

Choose **Settings > Appearance and language**. The interface updates in place and preserves the current session. If a native launcher message remains in the old language, restart the desktop application. The saved UI choice takes precedence over the installer-language file.

20. User-Facing Tool Reference

You normally request these operations in natural language; the names below explain what the model can call.

Available in every work mode

Tool	Capability
read_memory	Read one complete memory layer.
save_memory	Save a durable fact to global, mode, or project memory.
web_search	Search current public web results.
web_fetch	Read a public HTTP/HTTPS page or supported document.
context_status	Inspect tokens, messages, images, pins, and compression.
pin_context_file	Pin a text file to the current chat.
unpin_context_file	Remove one pin.
list_project_documents	List readable documents in the project.
read_project_document	Read text, Markdown, PDF, DOCX, JSON, YAML, or CSV.
list_skills	List optional skill metadata.
read_skill	Load one selected SKILL.md.
export_document	Export Markdown, DOCX, or PDF.
list_dir, read_file	Browse directories and read text with line ranges.
search_files, find_files	Fast literal/regex search and file globs.
write_file, apply_patch	Create or atomically edit text files.
make_directory, move_file, delete_file	Structured filesystem changes with task rollback.
list_task_changes, undo_task_changes	Inspect or revert the current task journal.
view_image	Attach a local image for visual analysis.

Added in Writing, Development, and Computer

Tool	Capability
task_plan_status	Read the persistent operational plan.
set_task_plan	Define ordered task steps.
update_task_step	Update progress and step results.
record_task_validation	Record a validation performed outside automatic checks.

Added in Development and Computer

Tool	Capability
repo_overview	Return the automatic repository map.
project_instructions	Read hierarchical project guidance for a path.
project_validation_profile	List detected/configured checks.
start_project_check	Start the primary or selected validation command.
find_symbol	Find declarations across supported programming languages.
document_symbols	List declarations and line ranges in one source file.
find_references	Find fast whole-word symbol uses across the project.
browser_open, browser_snapshot	Open and semantically inspect an isolated Edge page.
browser_fill, browser_click, browser_press, browser_wait	Interact with fresh element refs and wait for UI changes.
browser_select, browser_hover, browser_scroll	Dropdowns, hover states, and scrolling.
browser_upload, browser_download	Transfer a local file through the page or save a download.
browser_viewport	Set a desktop, tablet, or mobile viewport.
browser_console, browser_network	Inspect browser diagnostics incrementally.
browser_screenshot, browser_close	Attach the page for Qwen vision or close the session.
git_status	Show branch and file status.
git_diff	Show working or staged diff.
git_commit	Create a local commit from selected/current-task files.
run_command	Run a short Bash, PowerShell, or cmd command.
start_command	Start a persistent background command.
poll_command	Read incremental process output/status.
send_stdin	Send input to a background process.
terminate_command	Terminate a process and its child tree.

Added in Computer

Tool	Capability
screenshot	Capture and attach the primary display.
get_screen_info	Read screen and image coordinate dimensions.
move_mouse	Move pointer to image coordinates.
click	Click or double-click with a selected mouse button.
scroll	Scroll at an optional image position.
type_text	Type or paste Unicode text.
press_key	Press a key or key combination.

21. Practical Request Examples

Discussion: Compare these two production approaches without introducing coding procedures.

Research: Research the current state of virtual production stages. Keep contradictory findings, show uncertainty, and export the final synthesis as PDF.

Writing: Read the project treatment, rewrite act two in the same voice, and save a DOCX.

Development: Inspect this repository, reproduce the reported failure, fix the root cause, run the relevant tests, and summarize what changed.

Memory: Remember for this project that final renders go to D:\Show\Delivery.

Pinning: Pin docs\ARCHITECTURE.md for this chat.

Computer: Take a screenshot, open the application settings, enable the requested option, and verify the result with another screenshot.

Export: Save your existing answer as PDF. Do not research again.

22. Operational Notes and Limitations

Permanent product boundaries

The following are **explicit non-goals and will never be added** unless the owner personally reverses this product decision:

- Language servers or an LSP runtime/distribution layer. The built-in lightweight symbol index is the intended code-navigation solution.
- A persistent interactive terminal as a primary workflow. Shell tools remain a backup.
- Parallel model agents, subagents, or multi-model orchestration. One local model owns the GPU and performs the work sequentially.
- One-million-token context profiles. Context remains within practical local GPU profiles.
- A general plugin host, MCP ecosystem, or broad third-party integration framework.

These items do not belong to Marvin. They are not deferred roadmap work.

- One large model occupies nearly the full GPU, so model inference is intentionally single-slot.
- Read-only tools and background processes can still run concurrently.
- Switching model or KV cache restarts the server and clears its transient prompt cache, but not chat history.
- Context numbers are estimates; exact tokenization and image cost depend on the model/template.
- Model output can be wrong. Verification tools improve confidence but do not guarantee correctness.
- Computer control is limited to the primary monitor and depends on visible UI state.
- Internet extraction may fail on login walls, scripts-only sites, CAPTCHAs, blocked downloads, or unsupported formats.
- Public skills should be reviewed before installation. Skill text guides the model and can influence its actions.
- Git commit is local unless the user explicitly asks for a push and the environment permits it.
- The application is a personal local tool. Maintain backups of irreplaceable project and session data.

23. Extended Workspace Operations

Reading full documents

Ask for a specific PDF page, DOCX block, Excel sheet/cell range, or CSV row range. The document reader returns continuation coordinates so the next part can be read explicitly instead of stopping at the first hundred rows. Excel can expose cached values or formulas; the reader does not recalculate Excel formulas.

Scanned PDF pages and diagrams can be rendered with `view_document_page` and inspected by the current vision-capable model. Existing Word documents can be changed through `edit_word_document`, retaining paragraphs, tables, and run styling.

Recovering original context

Compression is adapted to Discussion, Research, Writing, Development, or Computer. All input chunks are processed, and intermediate summaries are cached. The original messages remain available through `search_chat_history` and `read_chat_history`, including content no longer in the active model context.

Research saves intermediate evidence notes. Interrupted synthesis can reuse those notes. Citation buttons open the original source text, while found but unloaded candidates remain visible without a credibility filter.

Restore points

Open **Results > Restore points** or use `/checkpoint name`. A restore point captures working files while excluding generated dependencies and model/runtime directories. Task snapshots also reconcile file changes made through model commands. **Restore** reports later conflicting edits instead of silently overwriting them, and a partial failure is not marked fully restored.

Portable projects

Use **Settings > Data and backups > Export project**. The ZIP contains project files, project memory and skills, conversation history, attachments, and decision references. **Import project** creates a new project directory and new chat IDs, remapping stored attachment paths and source-chat links.

This project archive is separate from the model/runtime backup. The latter still acts as an online-first fallback and can be used explicitly for offline setup.

Source development and releases

End users install Python 3.12 and the normal Setup.exe; Node.js is not a runtime prerequisite. Source developers build the frontend with `npm --prefix frontend ci` and `npm --prefix frontend run build`. Python serves the compiled `ui_dist` directory. Windows package versions are fixed in `requirements-windows-py312.lock`.

The old Gradio surface remains available for compatibility diagnostics with `MARVIN_LEGACY_UI=1`; the new workspace is the default.